🧩 Maze Madness Extension 4 — Complete the Maze

Topic: Add Two AND Conditions · **Course:** Maze Madness · **Level:** Extension (Advanced) · **Time:** about 55 minutes

The agent follows a redstone trail through a 3D maze with **four rules**, but the solver is **not finished**. At two junctions two signals fire at once and it stops — add **two AND rules**, then drive the agent to the goal.

Chat: `l` turn left, `r` turn right, `run` start solver, `rl` bring agent back.

1 · Read the Four Rules 🎛️

Each rule checks **one** signal and does **one** action.

```
if agent.detect(REDSTONE, LEFT):
    agent.turn(RIGHT_TURN)
elif agent.detect(REDSTONE, RIGHT):
    agent.turn(LEFT_TURN)
elif agent.detect(REDSTONE, DOWN):
    agent.move(UP, 1)
elif agent.detect(REDSTONE, UP):
    agent.move(DOWN, 1)
```

Fill the table from the code. You will reuse these actions.

Signal	Action
redstone **left**	
redstone **right**	
redstone **below** (down)	
redstone **above** (up)	

A left signal makes the agent turn *right*; a signal below makes it move *up*. Why might the maze pair a signal on one side with an action to the other?

2 · Find the Two Stuck Junctions 🔍

Most junctions have **one** signal on, so one rule fires. Two junctions have **two** signals on at once — there the four rules are not enough and the agent stops.

Type `run` and watch where it stops. Look around the agent.

Tick the signals on at each stuck junction.

	left	right	below	above
Junction 1				
Junction 2				

Only the first matching `if` / `elif` branch runs on each pass. Why does that leave the second signal unused?

3 · AND means BOTH 🧠

An **AND** is true **only when both** halves are true.

```
if agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, DOWN):
    ...
```

(Example only — yours may differ.) Left on but NOT below: is the whole condition true? Below on but not left?

Look at Junction 1 in your table. Write the AND condition true only at that spot, then the action for each signal in the order you want them.

4 · Add the Two New Rules 🛠️

Open the **M002 Complete the Maze** world and find the four rules. Add **two** `elif` **branches**, one per stuck junction, using the signals **you** found.

Junction 1 — write the whole branch: the AND condition, then the action for each signal.

```
elif agent.detect(REDSTONE, ______) and agent.detect(REDSTONE, ______):
    agent._________(__________)
    agent._________(__________)
```

Junction 2 — write the whole branch from scratch.

Both new rules must be checked *before* the four single-signal rules. Why? What happens at a stuck junction if a plain single-signal rule is checked first?

5 · Spot the Bug 🐛

Rewrite each block, then explain the fix. (Signals here are examples.)

Bug A — should fire **only when both** signals are on, but fires when **either** one is, so it triggers at wrong spots.

```
elif agent.detect(REDSTONE, LEFT) or agent.detect(REDSTONE, DOWN):
    agent.turn(RIGHT_TURN)
    agent.move(UP, 1)
```

Write the fixed code:

Why was it wrong?

Bug B — a junction needs **two** actions, but this rule does one, so the agent solves half and sticks again.

```
elif agent.detect(REDSTONE, RIGHT) and agent.detect(REDSTONE, UP):
    agent.turn(LEFT_TURN)
```

Write the fixed code:

Why was it wrong?

6 · Show Your Work 📸 🎥

The goal is start to the **very end** — past both stuck junctions and every plain turn. Type `run`, watch where it stops, fix your two rules, run again until it reaches the goal.

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Fill the blanks:

> Today I built _____.
>
> I built it using this code: _____.
>
> In this code I used _____.
>
> The hardest part was _____.
>
> That part was hard because _____.
>
> The most fun part was _____.
>
> Something new I learned was _____.

Write your lines here, then say them in your video.

Submit ✅

Send this worksheet + your video to teacher on KakaoTalk.